

Universidad de Antioquia

Departamento de Ingeniería de Sistemas
Arquitectura de Software

Laboratorio 2

**Entorno colaborativo de integración y
despliegue continuo con Spring Boot**

Estudiante: David Arango Pineda
C.C. 1000758519

Profesor: Diego Botia

Octubre de 2026

1. Introducción

En este laboratorio armé un pipeline de CI/CD para una aplicación hecha con Spring Boot. La aplicación es una API que devuelve datos aleatorios (naciones, monedas y aviación) usando la librería Faker. Cada vez que subo cambios a GitHub, el pipeline corre las pruebas, revisa la calidad del código en SonarCloud, genera una imagen de Docker, la sube a Docker Hub y la despliega en Render.

Para la entrega obligatoria hice lo que pedía el enunciado: conecté el pipeline a un proveedor de nube (elegí Render porque tiene plan gratis), modifiqué el `build.yml` para que despliegue, y trabajé con ramas y pull requests hacia `main` (trunk based). Aquí está el paso a paso y las pruebas del backend en la nube.

Direcciones del proyecto:

Recurso	Dirección
Repositorio en GitHub	https://github.com/DavidAraangoPineda/lab2p2026
Imagen en Docker Hub	https://hub.docker.com/r/davidarango6/lab2p2026
Proyecto en SonarCloud	https://sonarcloud.io/project/overview?id=DavidAraangoPineda_lab2p2026
Aplicación en Render	https://lab2p2026-latest.onrender.com

2. Cómo funciona el pipeline

El archivo `.github/workflows/build.yml` tiene cuatro trabajos (jobs). Cada uno depende del anterior con `needs`, así que si uno falla los demás no se ejecutan.

Trabajo	Qué hace
tests	Corre las pruebas unitarias con <code>mvn -B test</code> .
sonar	Compila con <code>mvn clean verify</code> y manda el análisis y la cobertura a SonarCloud.
build	Genera el JAR, lo sube como artefacto y publica la imagen en Docker Hub.
deploy	Le avisa a Render (con un deploy hook) para que descargue la imagen nueva. Solo corre en la rama <code>main</code> .

Además agregué la rama nueva y los pull requests en los disparadores, como pide el enunciado:

```
on:
  workflow_dispatch:
  push:
    branches:
      - main
      - feature/deploy-render
  pull_request:
    branches:
      - main
```

3. Paso a paso

3.1 Probar la aplicación en mi computador

Antes de tocar el pipeline corrí `./mvnw test` para ver que todo funcionaba. Pasaron las 7 pruebas del controlador. También agregué `server.port=${PORT:8080}` en el `application.properties`, porque Render le asigna el puerto a la aplicación con una variable de entorno llamada `PORT`.

3.2 Docker Hub

Creé mi cuenta (usuario `davidarango6`) y un repositorio público llamado `lab2p2026`. Después generé un token de acceso con permisos de lectura y escritura. Usé el token y no mi contraseña, porque es más seguro y se puede borrar si hace falta.

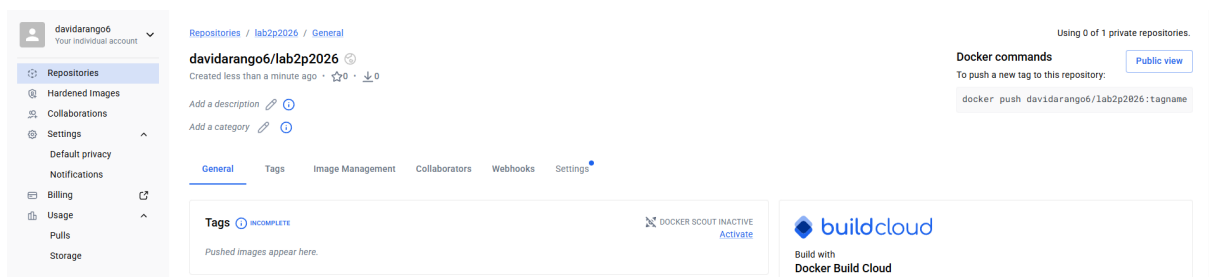


Figura 1. Repositorio creado en Docker Hub.

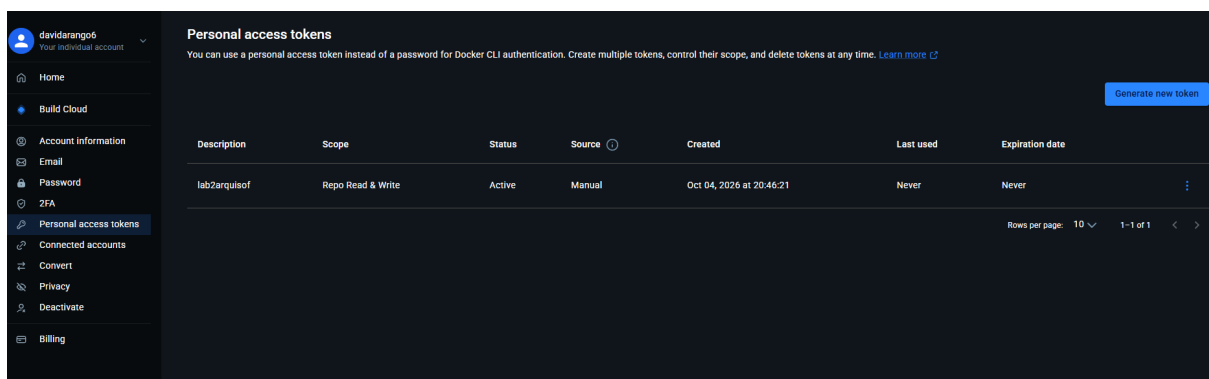


Figura 2. Token de acceso de Docker Hub.

3.3 Secrets en GitHub

En Settings, Secrets and variables, Actions guardé las claves que necesita el pipeline: `DOCKER_USERNAME`, `DOCKER_PASSWORD` (el token), `SONAR_TOKEN` y después `RENDER_DEPLOY_HOOK_URL`. Así no quedan escritas en el código.

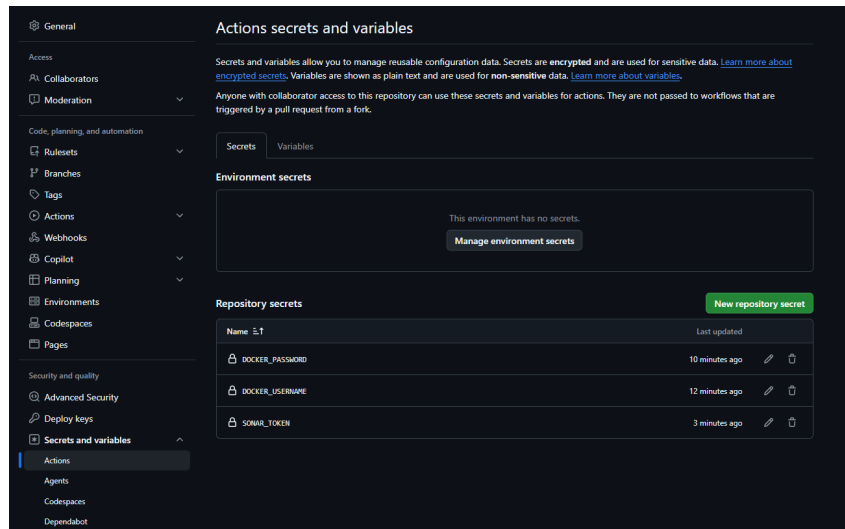


Figura 3. Los primeros tres secrets en el repositorio.

3.4 SonarCloud

Entré a SonarCloud con mi cuenta de GitHub, importé el repositorio (el plan gratis sirve para repositorios públicos) y apagué el Automatic Analysis, porque el análisis lo hace GitHub Actions. Luego generé un token y lo guardé como secret. Los datos que puse en el pom.xml y en el workflow son:

```
sonar.projectKey      = DavidAraangoPineda_lab2p2026
sonar.organization    = davidaraangopineda
sonar.host.url        = https://sonarcloud.io
```

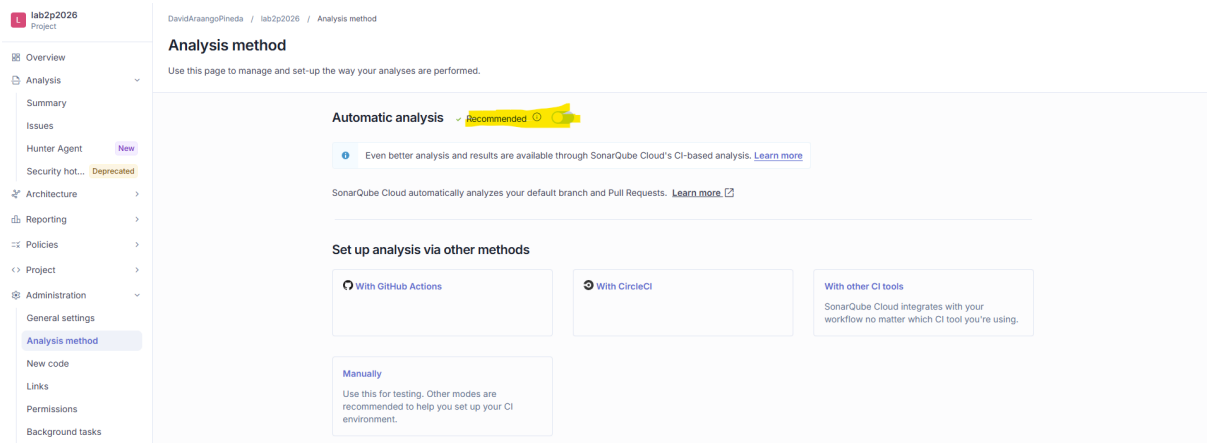


Figura 4. Automatic Analysis apagado en SonarCloud.

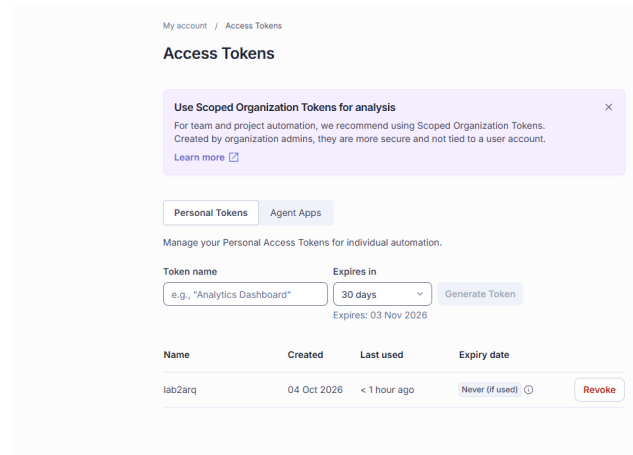


Figura 5. Token de SonarCloud.

3.5 Rama nueva y trabajo de despliegue

Creé la rama feature/deploy-render a partir de main y ahí agregué el trabajo que despliega en Render:

```
deploy:
  needs: build
  name: Deploy to Render
  if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'
  runs-on: ubuntu-latest
  steps:
    - name: Trigger Render deploy
      env:
        RENDER_DEPLOY_HOOK_URL: ${ secrets.RENDER_DEPLOY_HOOK_URL }
      run: curl --fail -X POST "$RENDER_DEPLOY_HOOK_URL"
```

La condición if hace que solo se despliegue cuando el cambio ya está en main. En las ramas y en los pull requests se corren las pruebas, Sonar y el build, pero no se despliega.

3.6 Primer error: la organización de Sonar

La primera vez que subí la rama, el trabajo de SonarCloud falló. En el log decía Organization key 'DavidAraangoPineda' does not exist. Yo había puesto el nombre que se ve en la página, pero la clave real de la organización es davidaraangopineda (todo en minúsculas). Lo confirmé con la API pública de SonarCloud, lo corregí en el pom.xml y en el build.yml y volví a subir. Esta vez pasó.

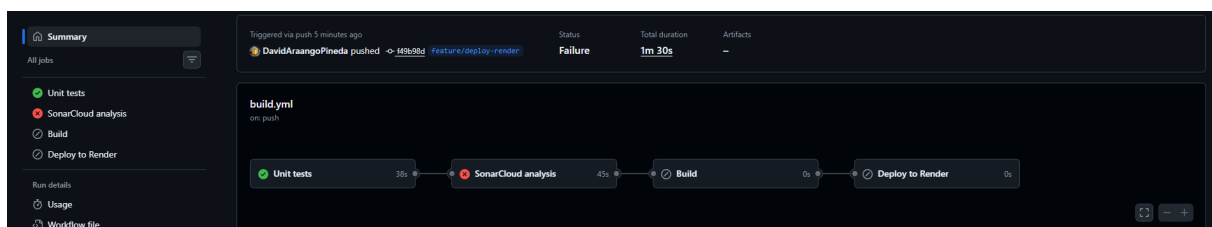


Figura 6. Ejecución fallida: error en el trabajo de SonarCloud.

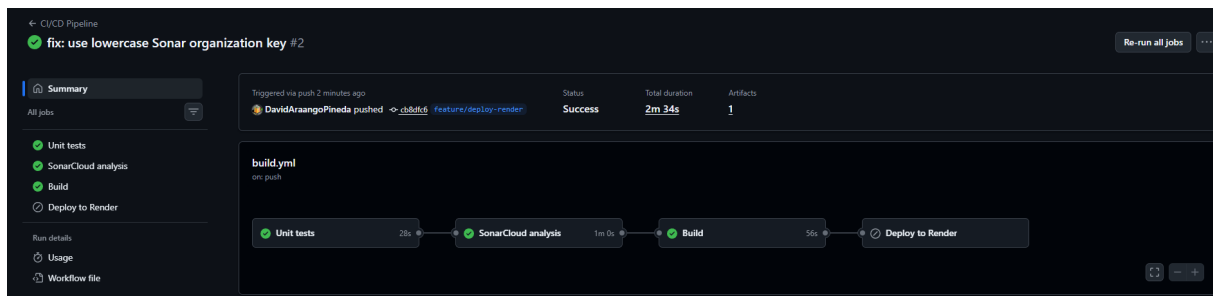


Figura 7. Después de corregir, pasan tests, SonarCloud y build. Deploy aparece omitido porque la rama no es main.

3.7 Imagen en Docker Hub

Cuando terminó el trabajo build, la imagen ya estaba en Docker Hub como `davidarango6/lab2p2026:latest`. El Dockerfile usa dos etapas: una con Maven para compilar y otra con Java 17 para ejecutar.

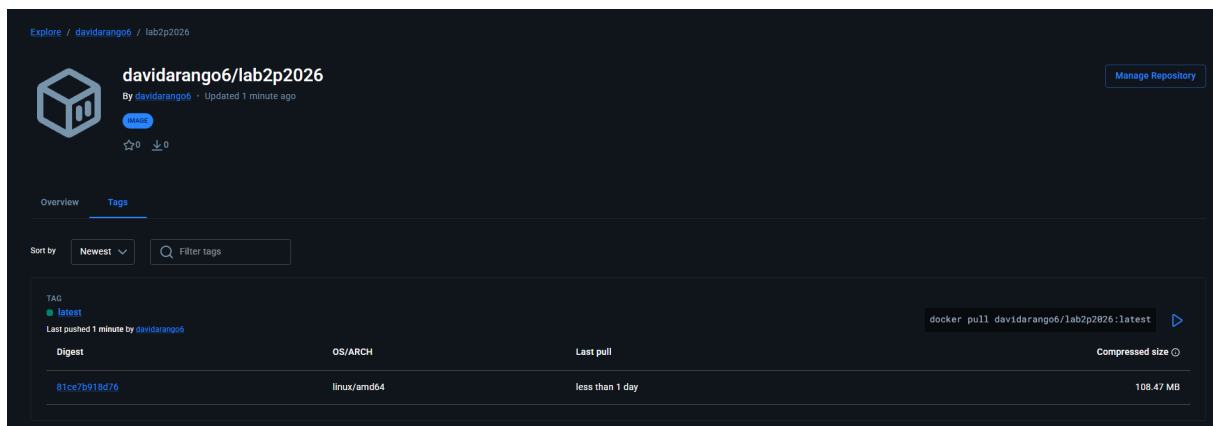


Figura 8. Imagen latest publicada en Docker Hub.

3.8 Render

En Render creé un Web Service con la opción Existing Image, plan gratis, usando la imagen `docker.io/davidarango6/lab2p2026:latest`. Como el repositorio de Docker Hub es público no hizo falta poner credenciales. Después copié el Deploy Hook (una URL secreta) y la guardé en GitHub como `RENDER_DEPLOY_HOOK_URL`.

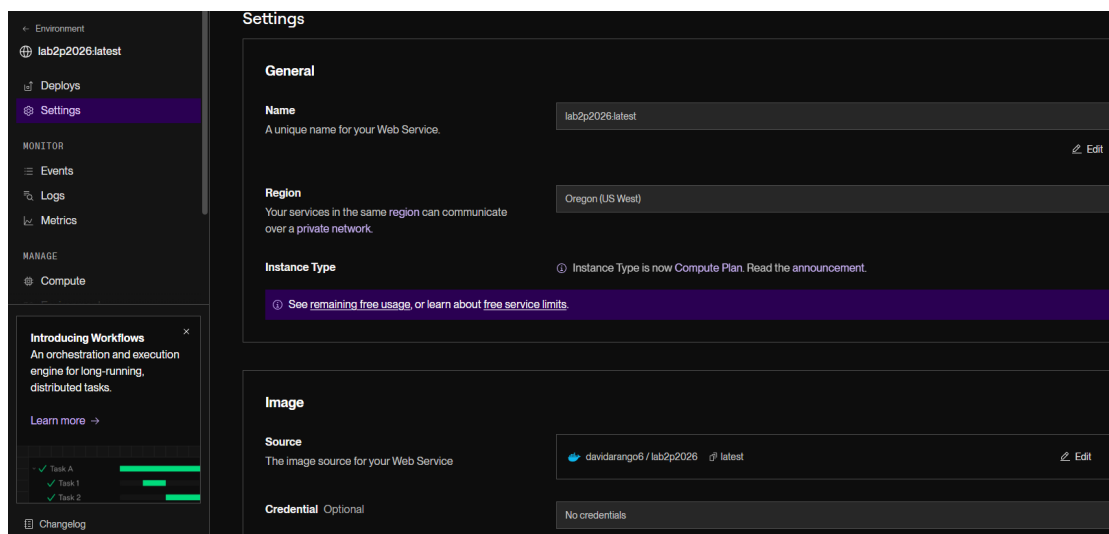


Figura 9. Configuración del servicio en Render.

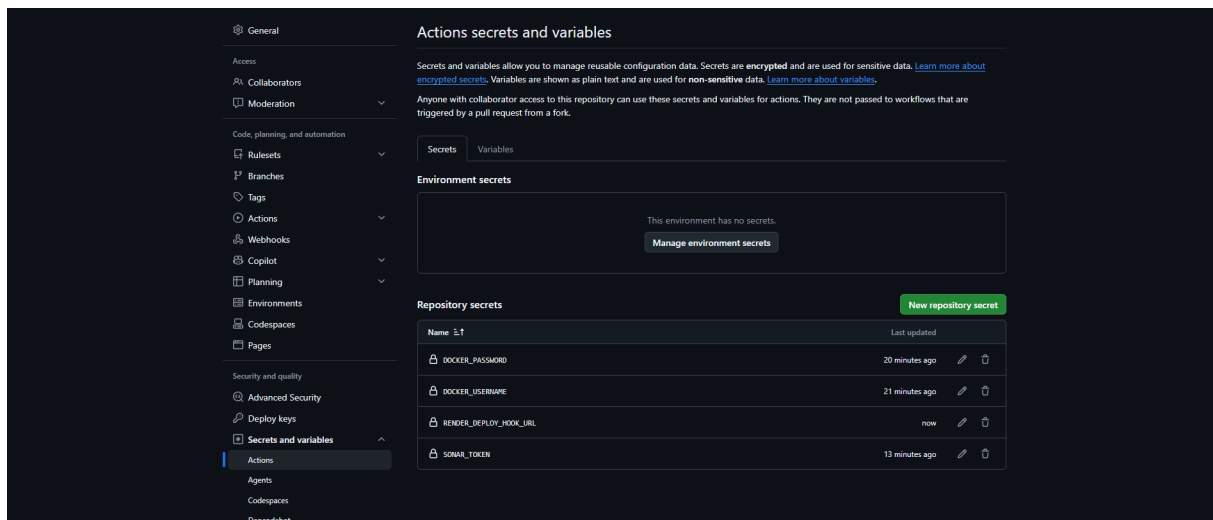


Figura 10. Los cuatro secrets del repositorio.

3.9 Pull request y segundo error: Quality Gate

Abrí el pull request #1 de feature/deploy-render hacia main. El pipeline corrió y SonarCloud dejó un comentario en el PR. La primera vez el Quality Gate falló por la condición "Security Rating on New Code" (salió C y se exige A o mejor).

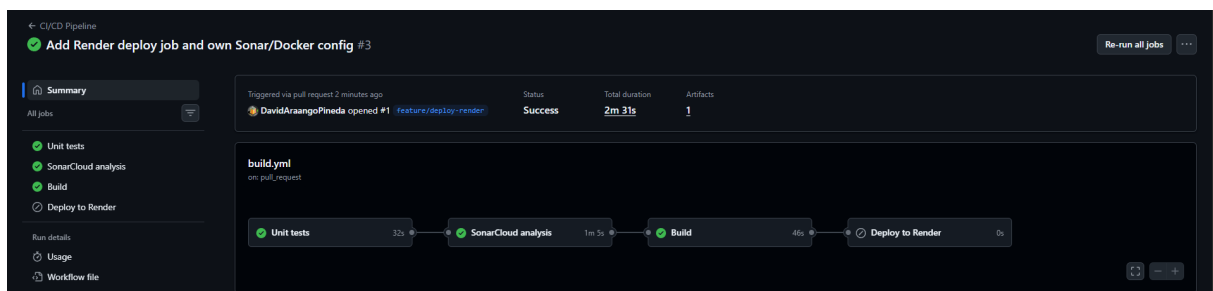


Figura 11. Pipeline del pull request.

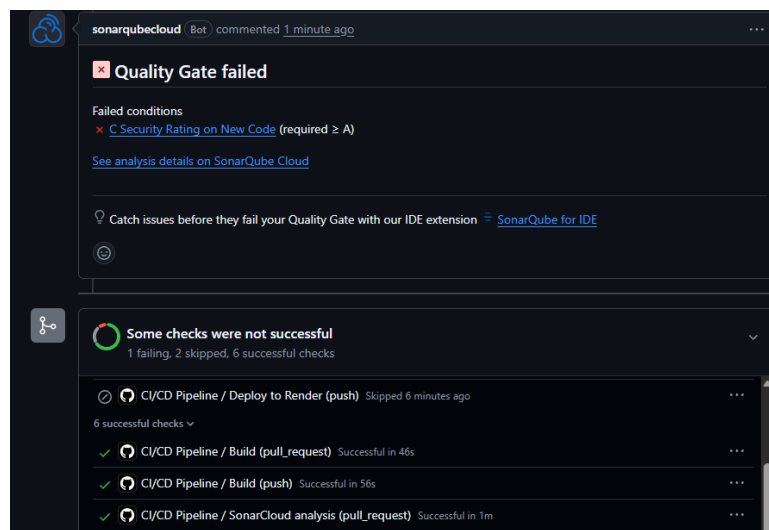


Figura 12. Quality Gate fallido en el pull request.

Revisé el detalle en SonarCloud y solo había una vulnerabilidad, la regla githubactions:S7637: la acción mr-smithers-excellent/docker-build-push@v6 estaba puesta con una etiqueta (v6), y una etiqueta se puede cambiar. Lo recomendado es usar el hash completo del commit, así siempre se ejecuta el mismo código. Lo cambié por:

uses: mr-smithers-excellent/docker-build-push@05b44e1dd9e03f78e8689bd1a61c750f17907ffe # v6

Con eso el Quality Gate pasó (0 incidencias nuevas y 0 hotspots de seguridad) y fusioné el pull request.

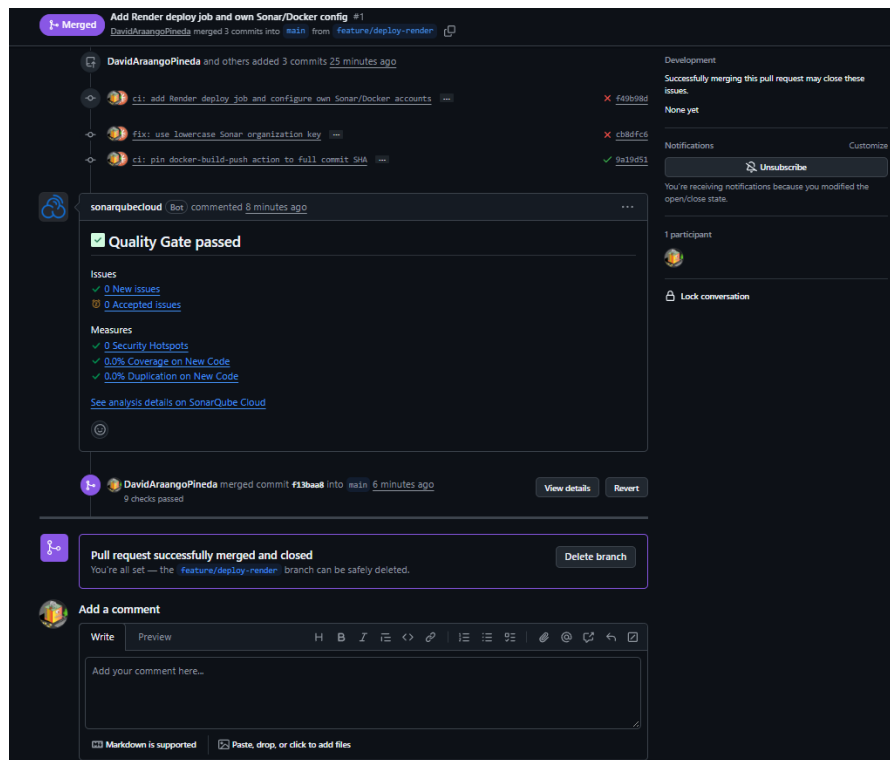


Figura 13. Pull request #1 fusionado con el Quality Gate aprobado.

3.10 Despliegue desde main

Al fusionar en main se disparó el pipeline otra vez, y ahora sí corrió el trabajo Deploy to Render. En Render se ve el nuevo despliegue como Live, con el disparador Deploy Hook.

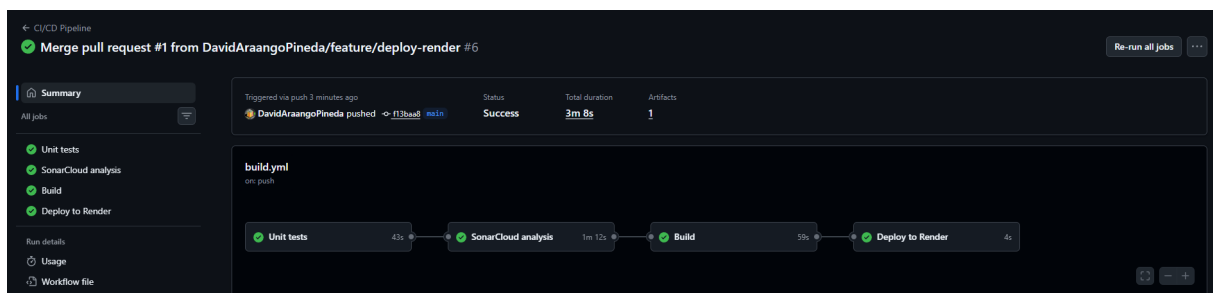


Figura 14. Pipeline en main con los cuatro trabajos en verde.

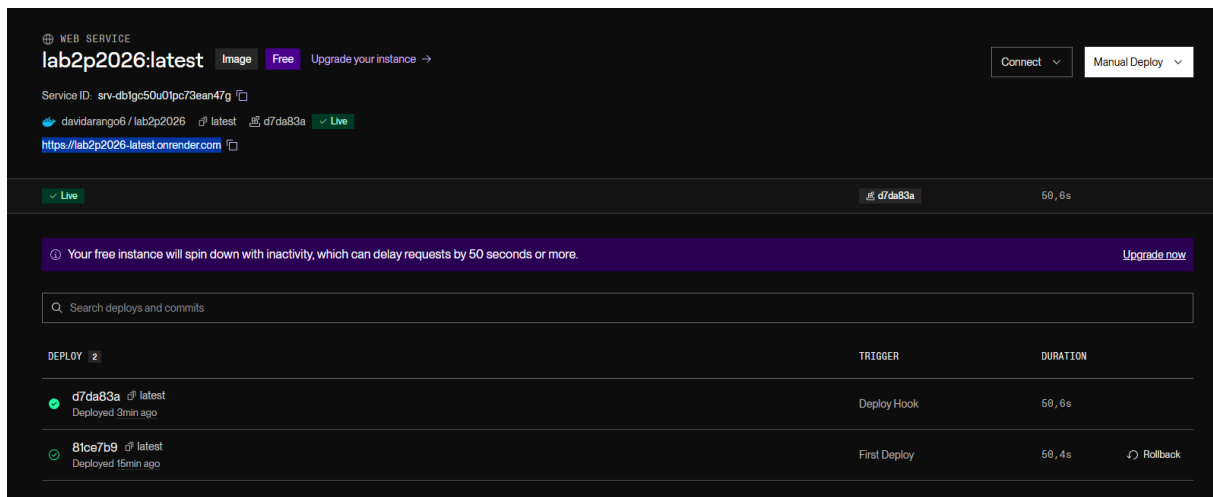


Figura 15. Render con el despliegue activo (Live).

3.11 Resultado en SonarCloud

El panel del proyecto muestra el Quality Gate aprobado, 93,0 % de cobertura (la mide JaCoCo) y 0,0 % de código duplicado.

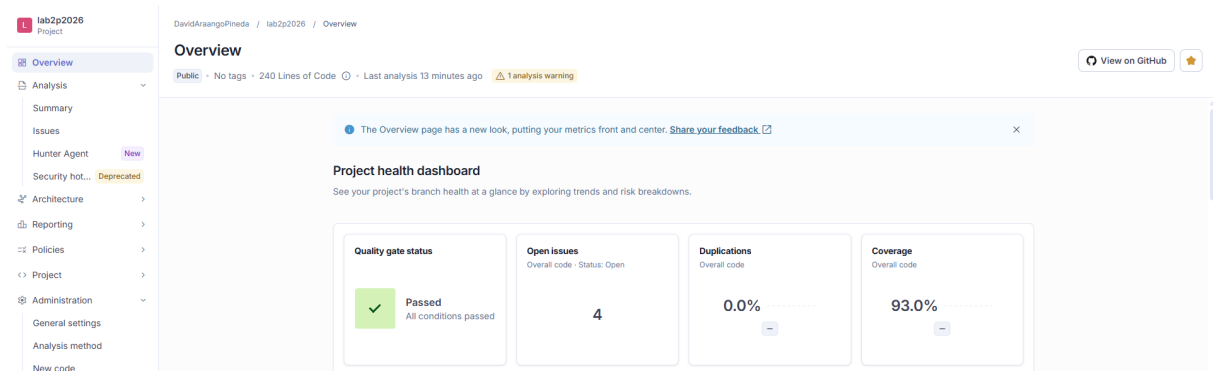


Figura 16. Panel del proyecto en SonarCloud.

4. Pruebas del backend en la nube

Probé con Postman las cinco rutas de la API, todas con GET, usando la dirección pública de Render (<https://lab2p2026-latest.onrender.com>). Todas respondieron 200 OK. Algunas tardaron varios segundos porque el plan gratis de Render tiene pocos recursos y suspende el servicio cuando no lo usan.

Ruta	Qué devuelve	Resultado
/	HEALTH CHECK OK!	200 OK
/version	The actual version is 1.0.0	200 OK
/nations	10 naciones en JSON	200 OK
/currencies	20 monedas en JSON	200 OK
/aviation	20 registros de aviación en JSON	200 OK

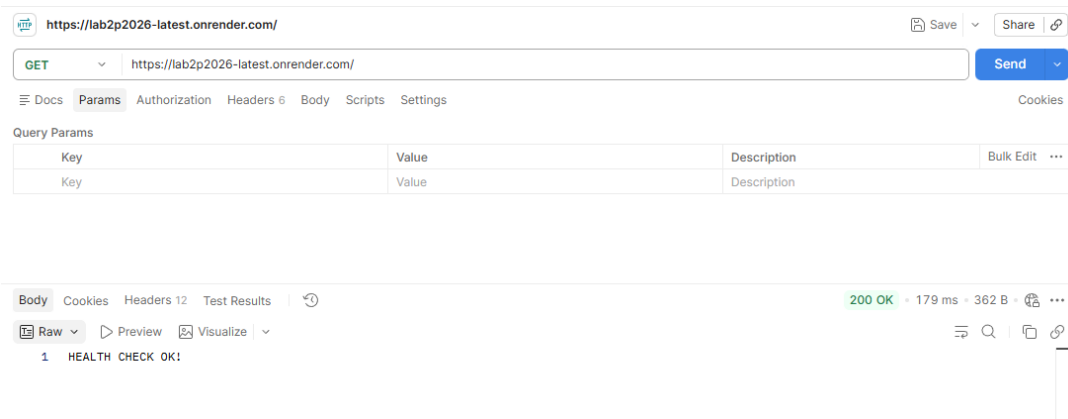


Figura 17. GET /

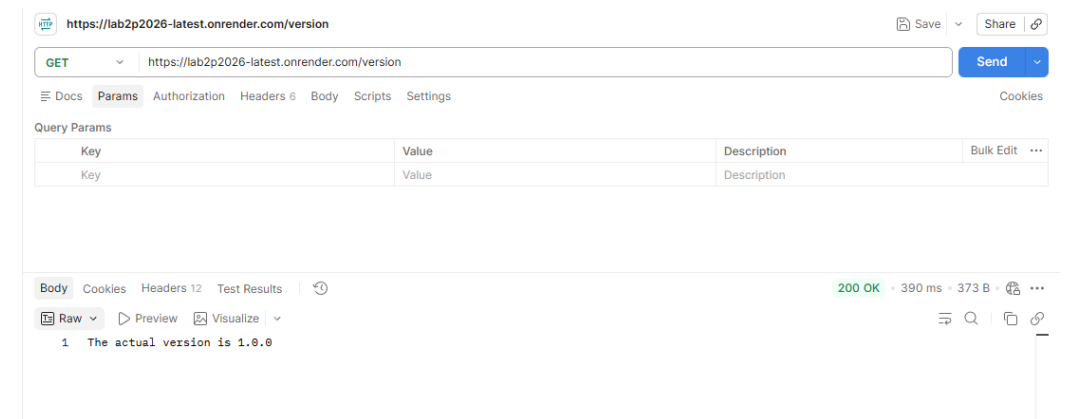


Figura 18. GET /version

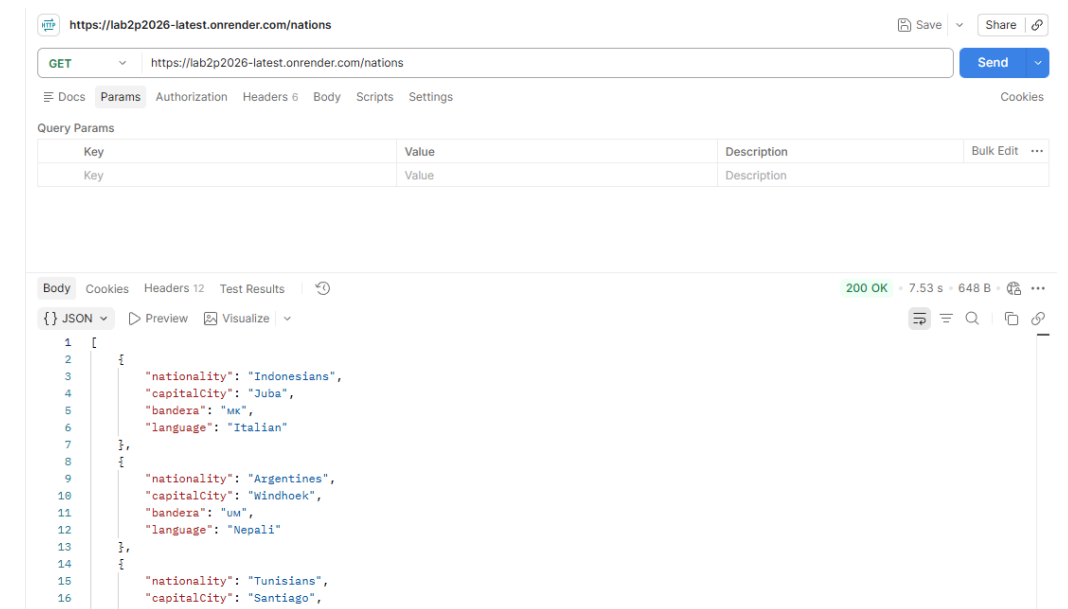


Figura 19. GET /nations

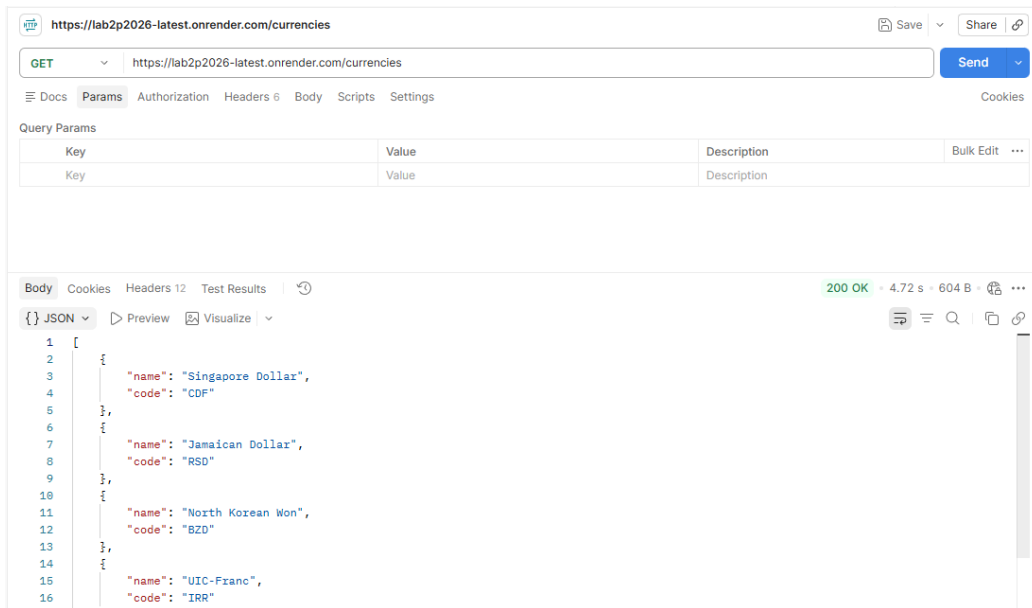


Figura 20. GET /currencies

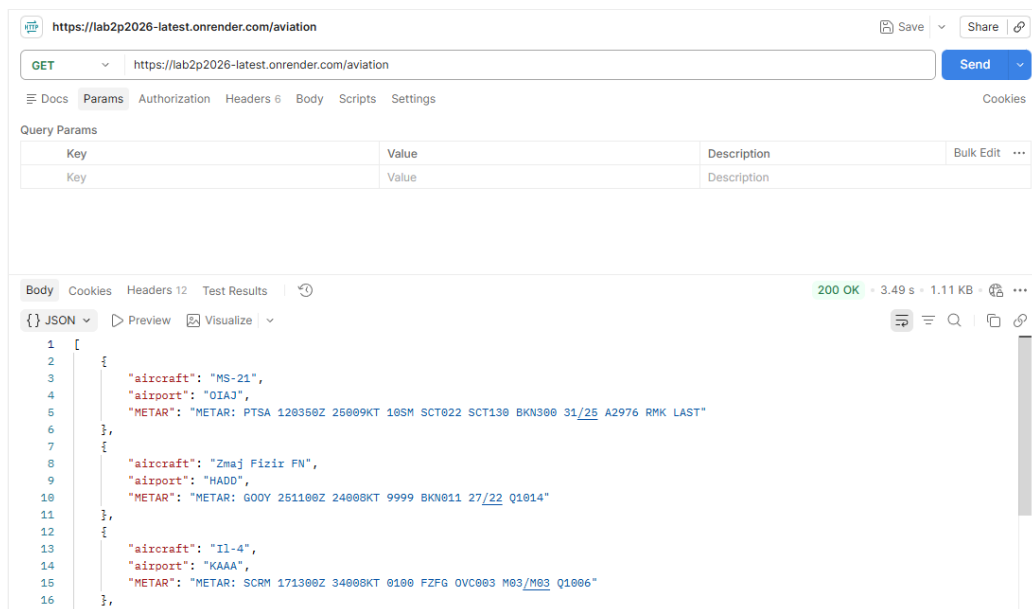


Figura 21. GET /aviation

5. Problemas que tuve

Problema	Por qué pasó	Cómo lo resolví
SonarCloud decía que la organización no existe	Usé el nombre visible y la clave va en minúsculas.	Puse davidaraangopineda en el pom.xml y en el build.yml.
Quality Gate fallido (Security Rating C)	La acción de Docker estaba con una etiqueta y no con el hash del commit.	La cambié por el hash completo.
Postman daba 404 en rutas que sí existen	Al copiar la URL se coló un salto de línea al final.	Lo borré y las rutas respondieron 200.
Respuestas lentas y un 502 una sola vez en Render	El plan gratis tiene pocos recursos y el servicio se duerme.	Volví a probar y respondió bien.

Problema	Por qué pasó	Cómo lo resolví
Los badges del README mostraban el proyecto del profesor	El README venía copiado con sus claves.	Los cambié por mi repositorio en la rama fix/informe-readme.

6. Segundo pull request

Para tener otro ciclo de integración creé la rama `fix/informe-readme`. En ella corregí los badges del README para que apunten a mi repositorio y a mi proyecto de SonarCloud, y agregué este informe en la carpeta `docs`, con un enlace desde el README. La integré a `main` con un pull request.

7. Conclusiones

Con este laboratorio entendí cómo se conectan las piezas de un pipeline: GitHub guarda el código, GitHub Actions corre los pasos, SonarCloud revisa la calidad, Docker Hub guarda la imagen y Render la ejecuta. Lo que más me ayudó fue ver los errores reales (la clave de la organización y la acción sin fijar), porque tocó leer los logs para encontrar la causa.

También vi la utilidad de trabajar con ramas cortas y pull requests: el pipeline revisa el cambio antes de fusionarlo, y solo lo que llega a `main` se despliega. Como mejora para más adelante, la API podría crear el objeto Faker una sola vez y no en cada petición, porque eso la hace lenta en el plan gratis de Render.